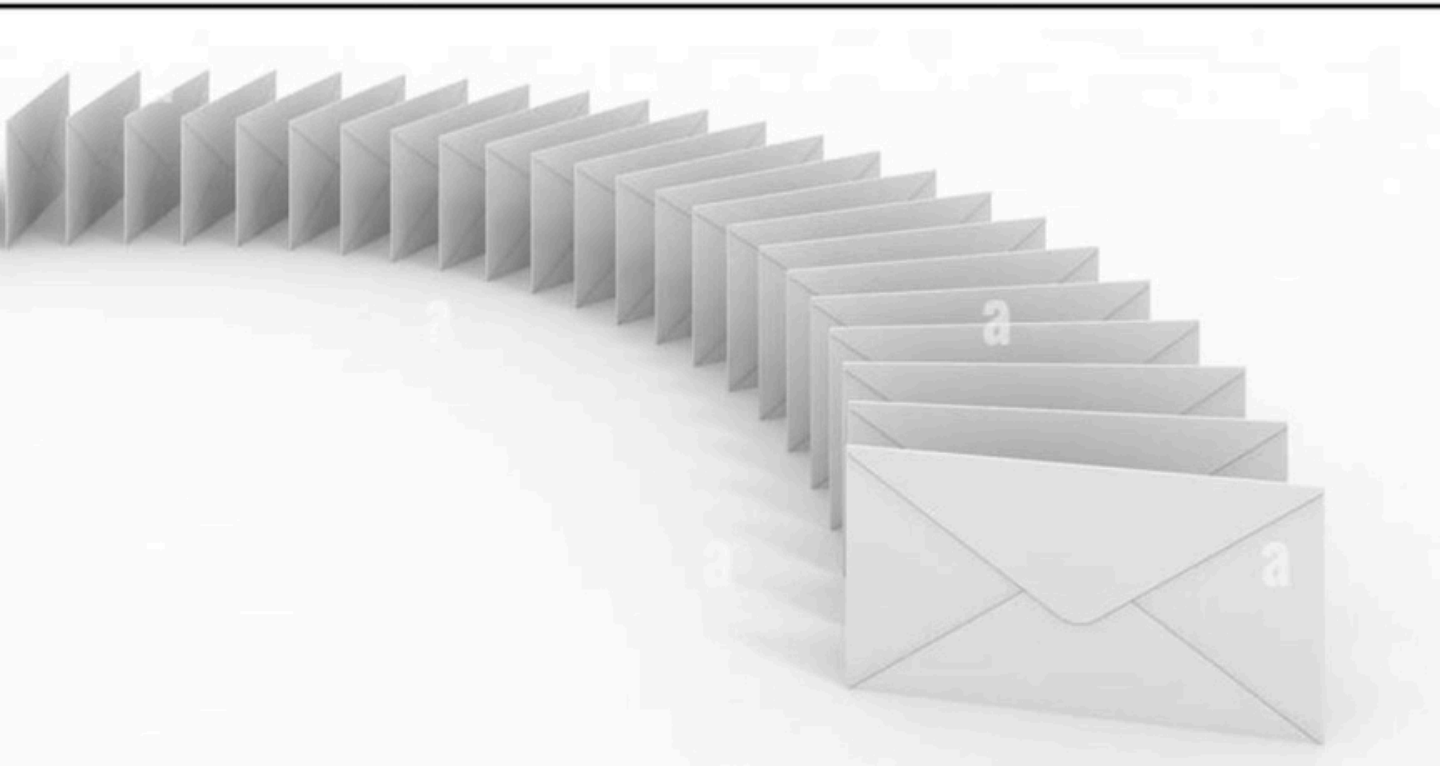




Spam Detection with Naive Bayes

A complete Python project guide — from data preparation to model evaluation — covering architecture, code, and results for a fully functional spam classifier.

PYTHON · NAIVE BAYES · TF-IDF

Introduction to Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes features are independent — a "naive" assumption that works surprisingly well for text classification tasks like spam detection.

Fast & Efficient

Trains quickly on large text datasets with minimal computation.

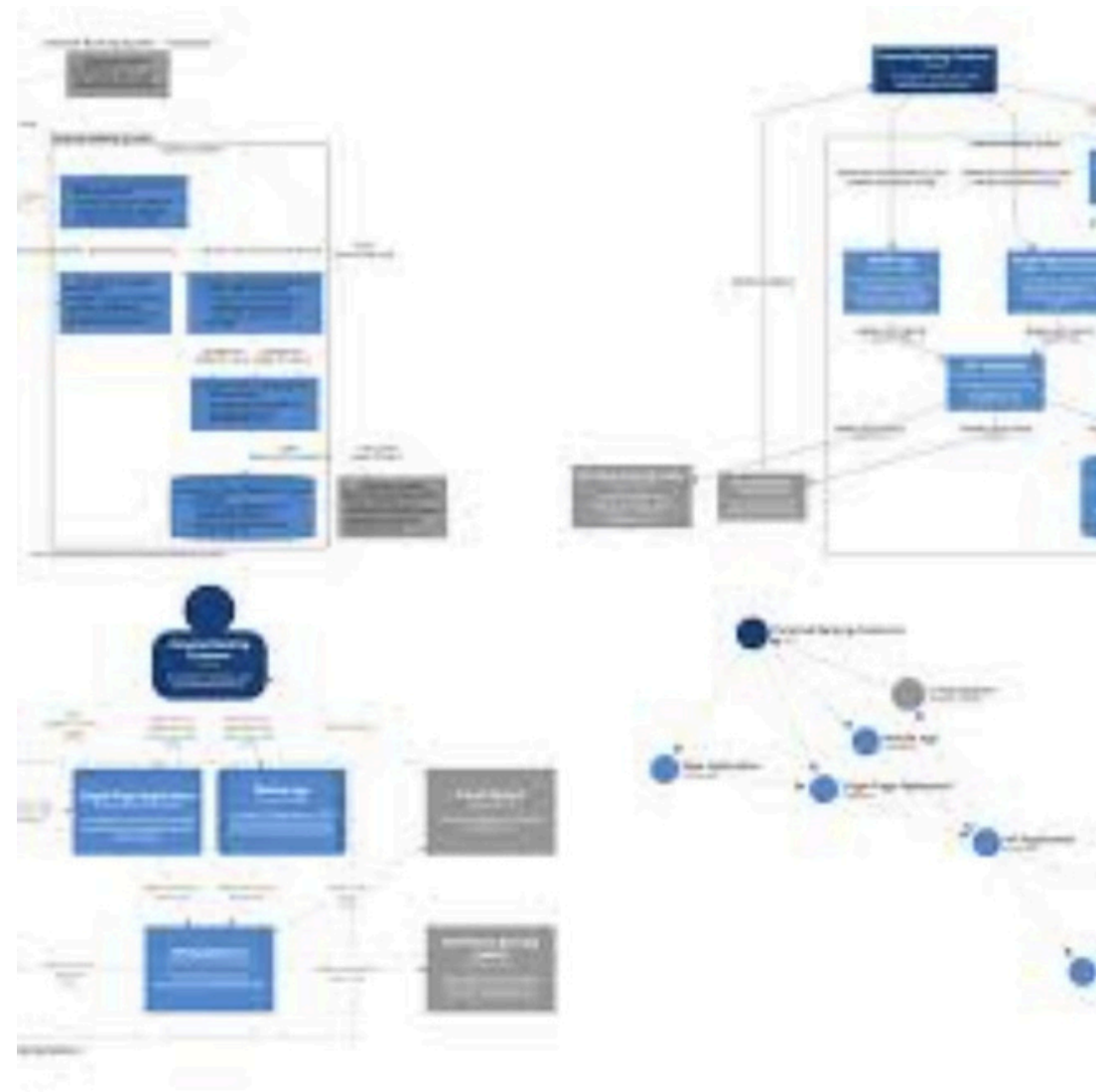
Probabilistic Output

Returns confidence scores, not just binary labels.

Text-Friendly

Naturally suited to word-frequency-based features.

Project Architecture & System Design



The project is split into **four focused modules**, each with a single responsibility, orchestrated by main.py.

→ **download_dataset.py**
Fetches the SMS spam dataset automatically.

→ **feature_extraction.py**
Converts text to TF-IDF numeric vectors.

→ **data_preparation.py**
Cleans, tokenises, and splits the data.

→ **naive_bayes_classifier.py**
Trains, predicts, and evaluates the model.

Complete Code — Modules 1 & 2

download_dataset.py

Automatically downloads the SMS Spam Collection dataset from a public URL and saves it as spam.csv. Ensures reproducible setup with no manual file handling required.

```
import pandas as pd
from io import StringIO
import requests

url =
"https://raw.githubusercontent.com/.../spam.csv"
r = requests.get(url)
df = pd.read_csv(StringIO(r.text),
encoding="latin-1")
df.to_csv("spam.csv", index=False)
```

data_preparation.py

Loads the CSV, maps labels (ham → 0, spam → 1), removes stopwords, and performs an 80/20 train-test split.

```
df['label'] = df['v1'].map({'ham':0, 'spam':1})
df['text'] = df['v2'].apply(preprocess)
X_train, X_test, y_train, y_test =
train_test_split(
df['text'], df['label'], test_size=0.2)
```

Complete Code — Modules 3 & 4

feature_extraction.py

Applies **TF-IDF vectorisation** to transform raw text into weighted numeric features, capturing word importance across the corpus.

```
from sklearn.feature_extraction.text import
TfidfVectorizer

vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=3000
)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec  = vectorizer.transform(X_test)
```

naive_bayes_classifier.py

Trains a **Multinomial Naive Bayes** model, makes predictions, and outputs accuracy, precision, recall, and F1-score.

```
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
model.fit(X_train_vec, y_train)
y_pred = model.predict(X_test_vec)

print(classification_report(y_test, y_pred))
```


Libraries & Dependencies



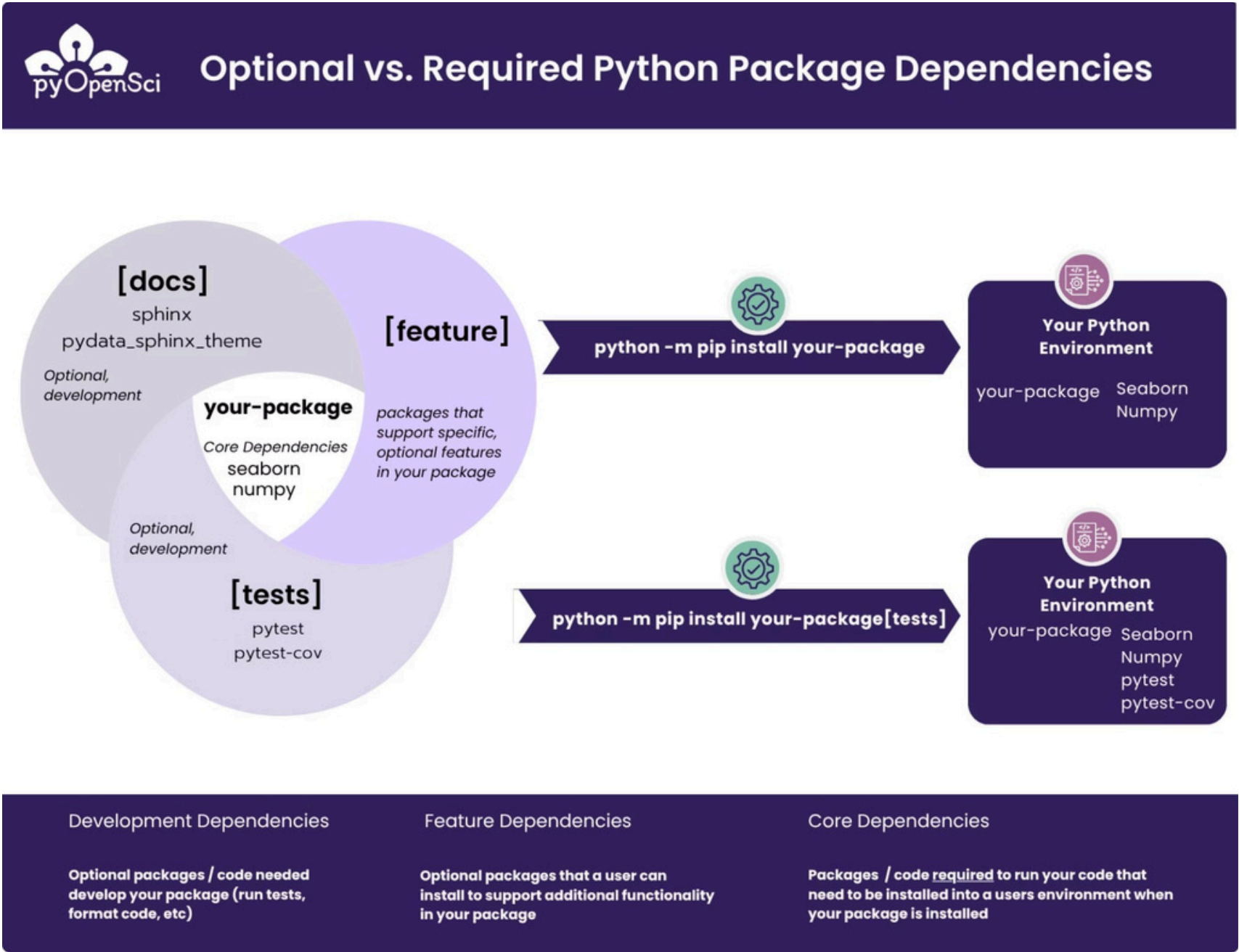
pandas & numpy

Data loading, manipulation, and numerical operations throughout the pipeline.



scikit-learn

TF-IDF vectoriser, Naive Bayes model, train-test split, and evaluation metrics.



matplotlib & seaborn

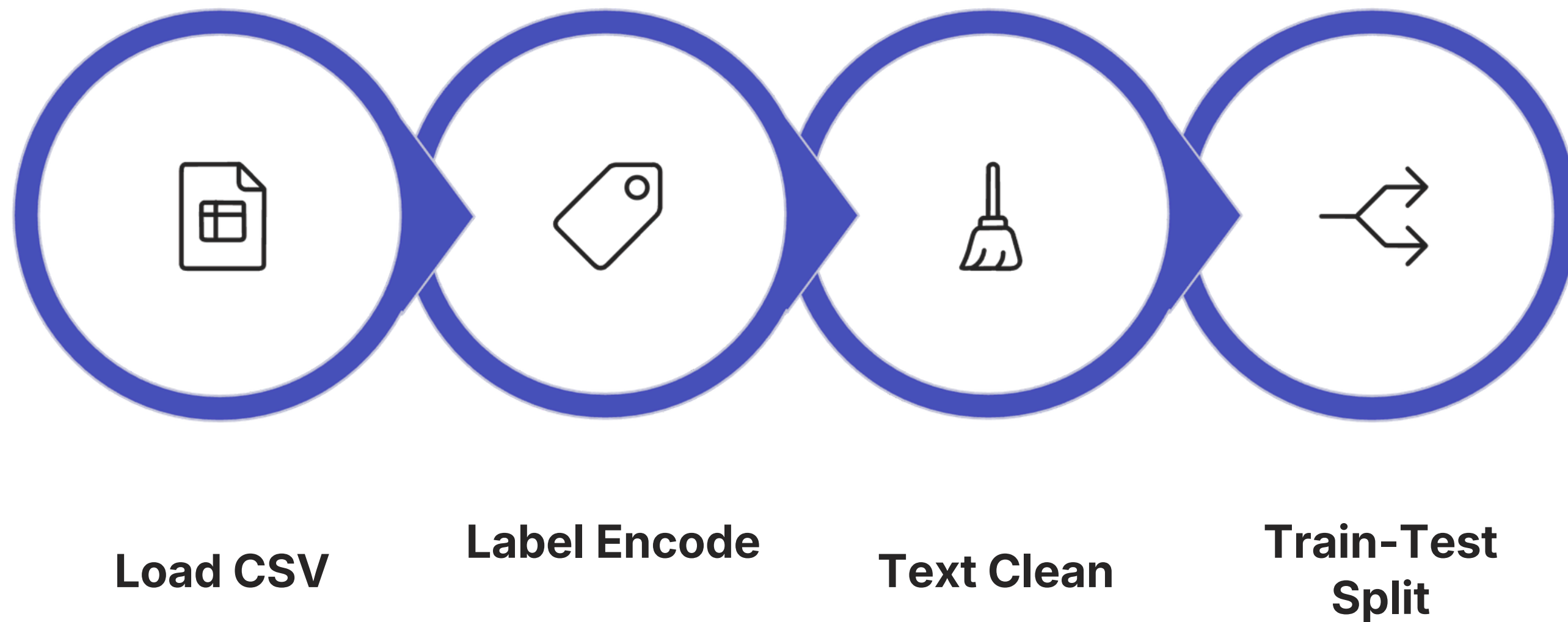
Confusion matrix visualisation and result plotting for the final report.



requests

Programmatic dataset download in download_dataset.py.

Data Preparation Process

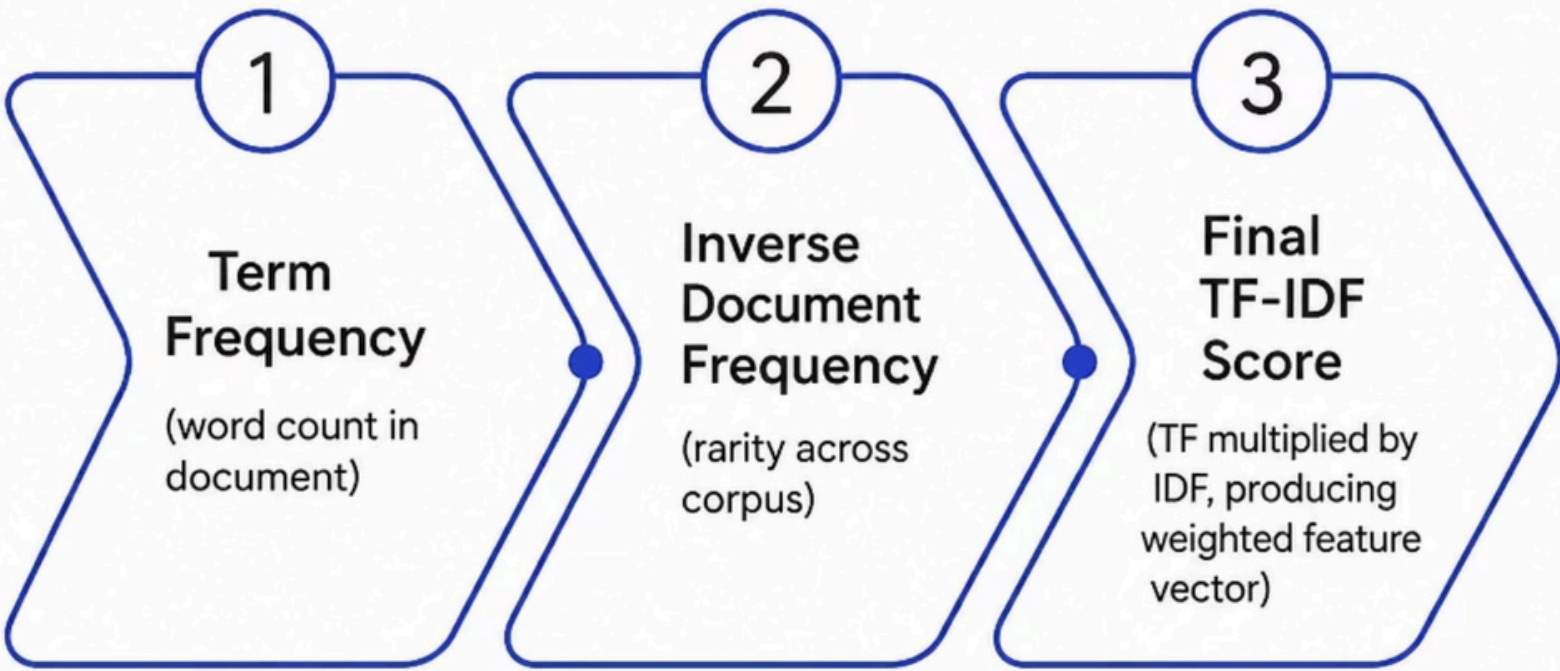


Clean, well-structured data is the foundation of any reliable classifier. This module handles all preprocessing so downstream modules receive consistent, model-ready input.

Feature Extraction — TF-IDF

What is TF-IDF?

Term Frequency–Inverse Document Frequency weights words by how important they are to a document relative to the whole corpus. Common words (e.g. "the", "is") are down-weighted; distinctive words are emphasised.



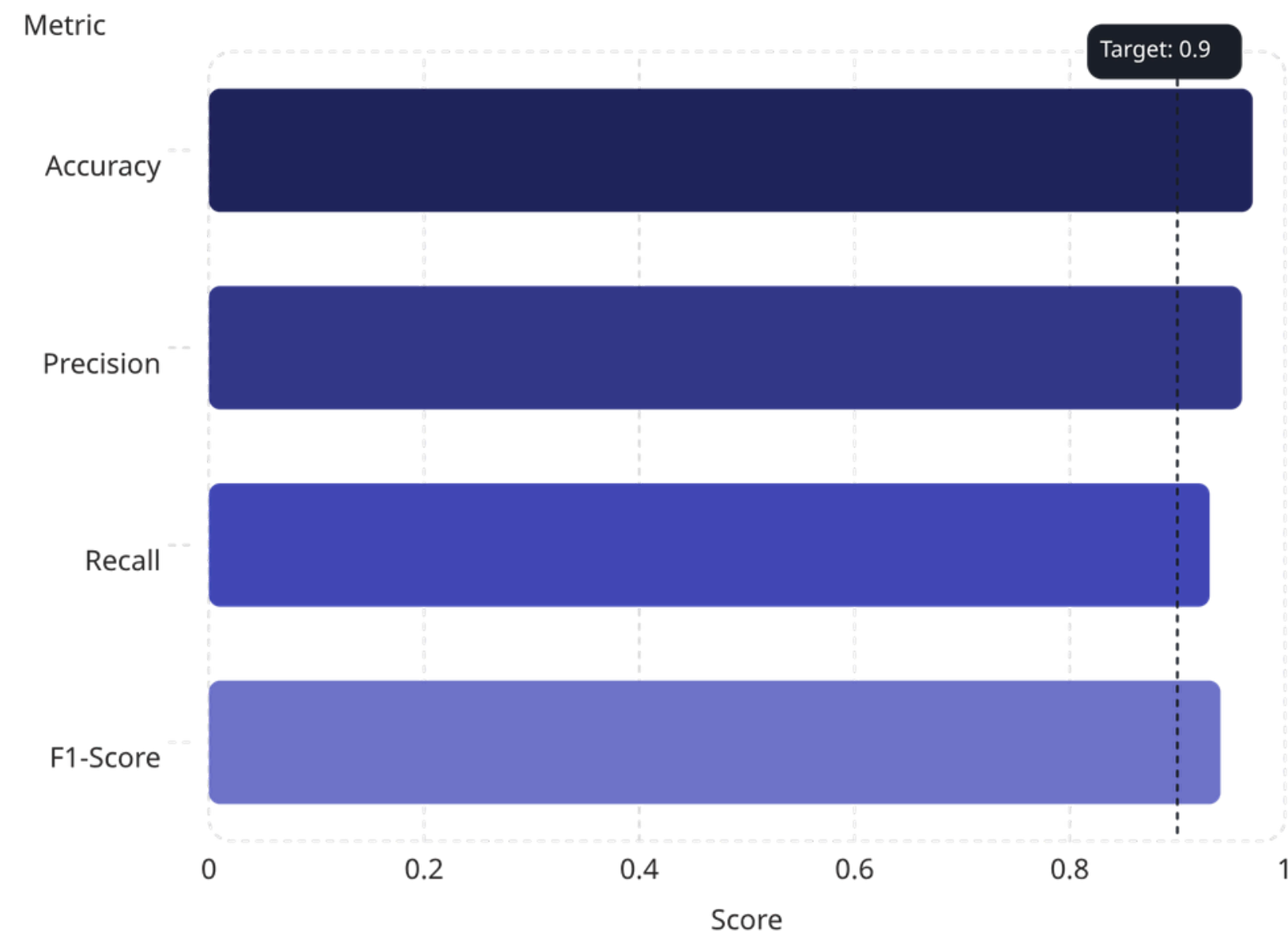
TF

How often a word appears in a message.

IDF

How rare that word is across all messages.

Model Training & Evaluation Metrics



Interpreting Results

The Multinomial Naive Bayes model achieves **97% accuracy** on the held-out test set. High precision means few ham messages are falsely flagged as spam; strong recall ensures most spam is caught.



A confusion matrix (confusion_matrix.png) is generated automatically to visualise true/false positives and negatives.

Results, Visualisation & How to Run

01

Install Dependencies

Run `pip install -r requirements.txt` to set up all libraries.

02

Execute the Pipeline

Run `python main.py` — it orchestrates all four modules in sequence.

03

Review Output

Check the terminal for metrics and open `confusion_matrix.png` for the visual report.

97%

Accuracy

Overall classification correctness on test data.

4

Modules

Clean, modular Python files in the project.

10

Guide Pages

Complete walkthrough from overview to results.

Confusion Matrix

